

Séance 1 - Concevoir une API HTTP

Au programme

- Positionnement (30 min, en tout début de séance)
- Le problème : une API n'est pas une liste de fonctions exposées en HTTP
- L'échelle de maturité de Richardson : les niveaux, et pourquoi viser le 2
- Ressources et collections : le modèle
- Concevoir les URIs : ce qui se fait, ce qui ne se fait pas
- Les verbes et leur sémantique : sûr, idempotent
- Les familles de codes de statut, et les pièges (201, 400 vs 422, 404 vs 403)
- Rappel HTTP fin : négociation de contenu, en-têtes conditionnels
- Le fil rouge : 3 sujets, une API par étudiant

Le problème : exposer des fonctions vs exposer des ressources

Le réflexe : prendre les fonctions du code et leur coller une URL.

```
1  POST /createReservation
2  POST /getReservationsByUser
3  POST /cancelReservationById
4  POST /updateSeatCount
```

Chaque équipe invente ses noms, ses conventions, ses codes de retour. Rien n'est prévisible : le client doit lire la doc pour **chaque** appel.

L'approche ressource : un petit nombre de **noms** (les ressources), un petit nombre de **verbes** standard (ceux d'HTTP), une **sémantique connue d'avance**.

L'échelle de maturité de Richardson

Une grille pour évaluer à quel point une API s'appuie sur HTTP.

Niveau	Ce qu'on ajoute	Exemple (billetterie)
0	rien : HTTP sert de tunnel, tout en POST	<code>POST /api + { "action": "reserver" } ou POST /createReservation</code>
1	des ressources , chacune son URI	<code>POST /reservations/42</code> pour lire, annuler, modifier
2	les verbes et codes HTTP	<code>GET / DELETE /reservations/42 , 201 , 404 , 409</code>
3	l' hypermédia : liens vers la suite	<code>"links": { "next": "/seances?page=2" }</code>

Leonard Richardson (2008), popularisé par Martin Fowler, « Richardson Maturity Model » (2010).

Richardson : pourquoi monter les niveaux

Plus on monte, plus le client (et le cache, le proxy, la bibliothèque HTTP) comprend l'API **sans lire sa doc**.

Niveau Ce que le client y gagne

- | | |
|---|---|
| 1 | une adresse stable par chose : on la désigne, on la relie, on la retrouve dans les logs |
| 2 | une sémantique connue : <code>GET</code> se cache et se rejoue, <code>DELETE</code> rejoué ne change rien, <code>4xx</code> $\neq$ <code>5xx</code> |
| 3 | les actions possibles sont dans la réponse : le client suit les liens au lieu de fabriquer les URLs |

Dans ce module : le niveau 2 partout, plus une touche de niveau 3 : les liens de pagination (`links.next` , séance 2). En TP, l'échelle sert de grille pour relire votre `api-design.md` .

Ressources et collections

Une **ressource** est une chose que l'API expose et qu'on peut nommer : une réservation, un adhérent, une séance de cinéma.

Une **collection** est une ressource qui contient d'autres ressources : l'ensemble des réservations, l'ensemble des adhérents.

- | | | |
|---|-------------------------|---|
| 1 | /reservations | la collection |
| 2 | /reservations/42 | un élément de la collection |
| 3 | /seances/7/reservations | les réservations d'une séance (sous-collection) |

On modélise **les choses**, pas les actions. « Annuler une réservation » n'est pas une ressource : c'est un changement d'état de la ressource `reservations/42` .

Concevoir les URIs

- **Noms au pluriel** pour les collections : `/ouvrages` , `/adherents` , `/cours`
- **Pas de verbe dans l'URI** : `/reservations/42` , pas `/getReservation?id=42`
- **Identifiant stable** dans le chemin : `/reservations/42`
- **Relations** en sous-ressource quand la vie de l'une dépend de l'autre : `/ouvrages/128/exemplaires`
- **Filtres, tri, pagination** en paramètres de requête, pas dans le chemin : `/ouvrages?`
`auteur=herbert&tri=titre&page=2`

Le chemin identifie **une** ressource ou **une** collection. Tout ce qui restreint, ordonne ou pagine une collection passe par la *query string*.

Les verbes et leur sémantique

Verbe	Sûr	Idempotent	Usage
GET	oui	oui	lire une ressource ou une collection
POST	non	non	créer un élément dans une collection, déclencher un traitement
PUT	non	oui	remplacer une ressource entière à un emplacement connu
PATCH	non	non	modifier partiellement une ressource (idempotent seulement si conçu pour)
DELETE	non	oui	supprimer une ressource

Sûr = ne modifie rien. **Idempotent** = rejouer la requête à l'identique laisse le serveur dans le même état qu'un seul appel. On s'en ressert en séance 4 pour la concurrence.

Les codes de statut : les familles

Famille	Sens	Exemples
2xx	succès	200 OK, 201 Created, 204 No Content
3xx	redirection / cache	304 Not Modified
4xx	le client s'est trompé	400, 401, 403, 404, 409, 422
5xx	le serveur a échoué	500, 502, 503

La famille se lit d'un coup d'oeil. Un client bien écrit branche sa logique de reprise sur le **premier chiffre** avant de regarder le détail.

Les codes de statut : les choix qui reviennent

- 201 Created après un POST réussi + en-tête Location vers la ressource créée
- 204 No Content quand il n'y a rien à renvoyer (souvent après DELETE)
- 400 Bad Request : la requête est mal formée (JSON invalide, champ absent)
- 422 Unprocessable Content : la requête est bien formée mais **sémantiquement** invalide (date de fin avant la date de début)
- 404 Not Found vs 403 Forbidden : « ça n'existe pas » vs « ça existe mais pas pour toi » - parfois on renvoie 404 exprès pour ne pas révéler l'existence

Un échange complet

```
1 POST /seances/7/reservations HTTP/1.1
2 Content-Type: application/json
3
4 { "sieges": [ "A12", "A13" ], "client": "c-901" }
```

```
1 HTTP/1.1 201 Created
2 Location: /reservations/42
3 Content-Type: application/json
4
5 { "id": 42, "seance": 7, "sieges": [ "A12", "A13" ], "statut": "confirmée" }
```

Le client crée dans une **collection** (`POST` sur `/seances/7/reservations`), le serveur répond **où** la ressource vit maintenant (`Location`). Aucune doc nécessaire pour comprendre l'échange.

HTTP : la négociation de contenu

- `Content-Type` décrit le corps **qu'on envoie** : `Content-Type: application/json`
- `Accept` décrit ce que le client **sait recevoir** : `Accept: application/json`
- Le serveur choisit une représentation et l'annonce dans le `Content-Type` de la réponse

Erreur classique : lire `Content-Type` sur une requête `GET` sans corps. C'est `Accept` qui porte la demande du client.

Les **en-têtes conditionnels** (`ETag` , `If-Match` , `If-None-Match`) servent au cache et à la concurrence : on les ouvre en séance 4, mais on les croise dès les premières réponses.

Le fil rouge : une API par étudiant

À partir d'aujourd'hui, chacun construit **sa** propre API HTTP, sur un sujet choisi en fin de séance.

- Chaque séance applique la notion du jour à **votre** API
- Le livrable de chaque TP est **produit en séance** et versionné dans votre dépôt
- **Projet individuel** : un sujet différent par étudiant
- Le module tient en 15h : **aucun développement n'est attendu hors séance**
- Le langage est **libre** : vous codez dans celui que vous maîtrisez déjà

Les 3 sujets

#	Sujet	Ce qui le caractérise
1	Bibliothèque municipale	catalogue, exemplaires, emprunts ; recherche et pagination d'un gros catalogue
2	Billetterie de cinéma	séances, plan de salle, réservation de places ; concurrence sur les sièges
3	Salle de sport	cours, plannings récurrents, réservation de créneaux ; quotas et liste d'attente

Domaines volontairement distincts : chacun rencontrera un **piège** différent (séance 4). Détail des attendus : `evals/projet-sujets.md` .

Choisir son sujet

- **Un étudiant par sujet** : trois sujets, trois étudiants
- Choix **immédiat** ; en cas de double préférence, on tranche en séance
- Le sujet choisi est acté dans le `README` de votre dépôt, **avant la fin de la séance**

Vous garderez ce sujet jusqu'à la soutenance (séance 10). Choisissez un domaine dont les règles métier vous parlent : vous allez les modéliser pendant 10 séances.

TP

TP - Modèle de ressources + initialisation du projet

Objectif : initialiser votre dépôt, choisir votre sujet, et concevoir le modèle de ressources et le jeu d'URIs de votre API.

- Handout : `supports/tp/session-01.md`
- **Livrable attendu** : dépôt initialisé (`README` , arborescence, lancement documenté) + `docs/api-design.md` contenant
 - la liste des ressources et collections de votre sujet (au moins 4 ressources liées)
 - le tableau URI x verbe x code de statut attendu pour les opérations principales
 - 2 à 3 choix de conception justifiés en une phrase
- Sujet fil rouge **acté et commité**